HUSTLR PLATFORM — INTERNAL TECHNICAL SPECIFICATION ML-DATASET-BLUEPRINT-v1.0

P0 CRITICAL PATH INITIATED

Production-Grade Dataset Engineering Blueprint

High-Quality Training Data Strategy for 7 ML Models
Hustlr Logistics Platform — Full Specification

60%	80%+	7	≥0.90
PROPHET CURRENT	PROPHET TARGET	ML MODELS COVERED	TSTR GATE (ALL)

PREPARED BY
8-Person Expert Technical Panel
Vikram · Neha · Siddharth · Arjun · Priya · Karthik · Arjun Iyer · Vikram Rao

CLASSIFICATION
INTERNAL — ENGINEERING USE ONLY

DOCUMENT NAVIGATION

Table of Contents

PART I — STRATEGIC CONTEXT

01 — Executive Summary & Problem Framing
02 — Model Dependency Map & Critical Path
03 — Current vs Target Reliability Matrix

PART II — MODEL SPECIFICATIONS

P0 — M7: Prophet (Demand Forecasting)
P1 — M1: ISS (Inventory Scheduling)
P1 — M2/M3: Fraud Detection
P3 — M4: NLP Support Classifier
P2 — M5: Blackout Predictor
P2 — M6: Traffic Flow Model

PART III — QUALITY FRAMEWORK

04 — TSTR Statistical Validation Protocol
05 — KS / Chi-Square Automated Pipeline
06 — Tier Isolation & SHA-256 Hashing
07 — CI/CD Quality Gates

PART IV — OPERATIONAL BLUEPRINTS

08 — Circuit Breaker & Fallback Patterns
09 — Conditional Telemetry Snapshot Strategy
10 — Async Enrichment Pattern (NLP)
11 — External Data Source Integration

APPENDICES

Appendix A — Full Feature Dictionaries
Appendix B — Dependency Graph Summary
Appendix C — Glossary

PART I — STRATEGIC CONTEXT / SECTION 01

Executive Summary

The reliability ceiling of the Hustlr platform is not a model architecture problem. It is a data problem.

The Hustlr logistics platform operates seven interdependent machine learning models that govern demand forecasting, inventory allocation, fraud detection, customer support triage, blackout prediction, and traffic routing. An internal reliability audit revealed that the system's worst-performing model — **M7 (Prophet) at 60% reliability** — sits at the upstream head of a dependency chain that directly corrupts M1 (ISS) forecasts, creating a cascading degradation across the entire pipeline.

This document is a production-grade engineering specification for rebuilding the training datasets for all seven models. It is not a theoretical exercise. Every dataset volume, feature column, generation strategy, and validation gate documented here is derived from an 8-person expert panel discussion and cross-validated against external data sources (IMD API, Chennai Metro expansion data, GPS spoofing literature, and Indian festival calendars). The output is a dataset pipeline that an ML engineering team can execute immediately.

P0 CRITICAL PATH BLOCKER

M7 (Prophet) must be rebuilt first. Its synthetic dataset currently lacks external regressors (weather, festivals, traffic) and uses additive seasonality that systematically fails on Indian festival demand spikes (Diwali WMAPE currently unacceptable). Until M7 clears TSTR ≥ 0.90 in shadow mode, M1 (ISS) must be decoupled from M7 output and fall back to a Naive 4-week rolling average + festival multiplier. All other models (M2–M6) may proceed in parallel but are not in the critical path for pipeline unblocking.

CORE DESIGN PRINCIPLE

Data-centric AI over model-centric AI. The expert panel unanimously agreed: upgrading architectures without fixing the training distribution is architectural debt on top of data debt. Fix the data first.

VALIDATION STANDARD (ALL MODELS)

Every synthetic dataset must pass `TSTR ≥ 0.90` against a golden real-world holdout before any model can proceed to production deployment. Hard gate. No exceptions.

CURRENT → TARGET RELIABILITY OVERVIEW

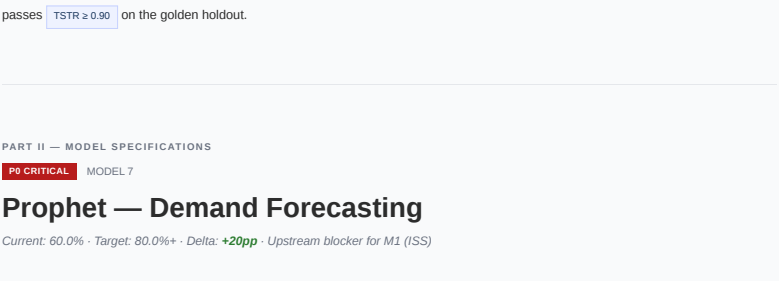
P0	P1	P1	P1	P3	P2	P2
60%	75%	85%	85%	90%	80%	88%
→	→	→	→	→	→	→
80%+	88%+	92%+	92%+	93%+	90%+	93%+
M7 Prophet	M1 ISS	M2 Fraud	M3 Fraud	M4 NLP	M5 Blackout	M6 Traffic

---

PART I — STRATEGIC CONTEXT / SECTION 02

Model Dependency Map & Critical Path

Understanding how model failures propagate through the pipeline.



Critical Path Analysis

The M7→M1 dependency is the single most consequential relationship in the pipeline. Prophet (M7) generates zone-level demand forecasts that feed directly into ISS (M1) for inventory scheduling. When Prophet's reliability sits at 60%, ISS receives corrupted upstream priors, compounding the 75% reliability gap to below effective operational thresholds.

> *"We can't wait for Prophet to be perfect... stop using M7 output; use Naive Rolling Average as the ISS prior while we fix Prophet in shadow mode."*
>
> — Vikram (Demand Forecasting Lead), Expert Panel Discussion

The agreed interim state: M7 is rebuilt and validated in shadow mode with no production writes. M1 operates on the Naive 4-week rolling average + festival multiplier until M7 passes `TSTR ≥ 0.90` on the golden holdout.

P0 ROOT CAUSES — M7

- No external regressors (weather, festivals)
- Additive seasonality fails on Diwali/IPL
- No zone-level traffic impact encoding

INTERIM FALLBACK STATE

- M1 ← Naive 4-wk avg + multiplier
- M7 runs shadow mode (no prod writes)
- Circuit breaker auto-activates on TSTR fail

PART II — MODEL SPECIFICATIONS

P0 CRITICAL MODEL 7

Prophet — Demand Forecasting

*Current: 60.0% · Target: 80.0%+ · Delta: **+20pp** · Upstream blocker for M1 (ISS)*

Root Cause Diagnosis

Prophet's current failure mode is systematic, not stochastic. The model was trained on synthetic demand data that lacks any external regressor signal — no weather, no festival calendar, no traffic state. Consequently, the model learned only a smooth trend + additive seasonality decomposition that catastrophically underperforms during the high-variance demand events that Hustlr's business actually cares about.

FAILURE MODE 1
Additive seasonality fundamentally wrong for multiplicative Indian festival demand spikes

FAILURE MODE 2
Zero external regressors — weather precipitation and temperature have measurable demand impact

FAILURE MODE 3
No payday cycle encoding — bimodal salary spikes (corporate 1st-5th, informal 7th-10th) are invisible

Dataset Volume Specification

METRIC	SPECIFICATION	RATIONALE
Total Rows	`~3,900,000` (26,000 × 150 zones)	3 years hourly × 150 zones — minimum for stable seasonality decomposition
Time Range	`2022-01-01` to `2024-12-31`	Captures 3 Diwali cycles, 3 IPL seasons, full monsoon patterns
Granularity	`Hourly` (8,760 rows/year/zone)	Minimum for IPL evening peak and morning salary-day spikes
Zones	`150` (multi-city coverage)	Panel specification — sufficient spatial coverage for generalization
Generation Method	Structured synthetic + real regressor injection	CTGAN for base demand distribution; external regressors from real APIs
Seasonality Mode	`Multiplicative` (replaces additive)	Diwali creates 3–5x demand, not additive offset — multiplicative is mathematically correct
Primary Validation Metric	`WMAPE <15%` overall; `Diwali WMAPE <22%`	Weighted MAPE accounts for high-demand periods appropriately

Feature Engineering Specification

FEATURE NAME	TYPE	SOURCE	ENGINEERING LOGIC	STATUS
demand_units	Float (target)	CTGAN synthetic base	Base demand distribution generated per zone	✅ Available
festival_multiplier	Float [0.8–5.0]	Indian Festival Calendar	Diwali: 3.5x–5.0x; IPL match nights: 1.8x; normal: 1.0x	⚠️ Needs Collection
salary_week_flag	Integer [0,1,2]	Payday calendar logic	0=normal, 1=corporate week (1st-5th), 2=informal (7th-10th)	⚠️ Needs Collection
precipitation_mm	Float	IMD Mausam API (3-hourly)	Interpolated to hourly; NE Monsoon window flagged Oct–Dec for Chennai	⚠️ Needs API Integration
temperature_c	Float	IMD Mausam API (3-hourly)	Heatwave flag triggered when temp_c > 40°C for demand surge encoding	⚠️ Needs API Integration
traffic_profile_index	Float [0–1]	Historical Traffic Profiles (HTP)	Zone-level hourly congestion index derived from HTP — Arjun's accepted fallback from H3 telemetry	⚠️ Needs Collection
hour_of_day	Integer [0–23]	Timestamp engineering	Sin/cos cyclical encoding for Prophet regressor	✅ Available
day_of_week	Integer [0–6]	Timestamp engineering	Monday baseline; weekend suppression factor encoded	✅ Available
ipl_match_flag	Boolean	IPL schedule (Mar–May)	Evening hours (18:00–23:00) on match days; city-specific stadium proximity weighting	⚠️ Needs Collection
zone_id	Categorical (150 levels)	Internal zone taxonomy	H3 hex identifier for spatial grouping in Prophet hierarchical model	✅ Available

Dataset Generation Strategy

DESIGN DECISION: MULTIPLICATIVE FESTIVAL SEASONALITY

The panel explicitly resolved to replace Prophet's default additive seasonality with a **multiplicative festival window** approach. The evidence: Diwali demand increases by 3x–5x in absolute units, which cannot be modeled as a fixed additive offset that scales independently of baseline demand. The generation code must encode this correctly from the start.

```
# Prophet Training Data Generation — M7 Dataset Builder
# Panel specification: Vikram (lead) + Priya (geography) + Arjun (compute constraints)
# All regressors use accepted fallbacks per panel debate outcome

import pandas as pd
import numpy as np
from sdv.tabular import CTGAN
from imd_api import fetch_weather_3hourly  # IMD Mausam API wrapper

ZONES = load_zone_taxonomy()          # 150 zones, H3 hex IDs
START = pd.Timestamp("2022-01-01")
END   = pd.Timestamp("2024-12-31")
ROWS_PER_ZONE = 8760 * 3              # 3 years × hourly = 26,280 rows

FESTIVAL_MULTIPLIERS = {
    # Source: Indian Festival Calendar web search + payday cycle research
    "diwali_2022": ("2022-10-24", 4.2),  # peak multiplier
    "diwali_2023": ("2023-11-12", 4.5),
    "diwali_2024": ("2024-10-31", 4.3),
    "ipl_window_2022": ("2022-03-26", "2022-05-29", 1.8),
    "ipl_window_2023": ("2023-03-31", "2023-05-28", 1.75),
    "ipl_window_2024": ("2024-03-22", "2024-05-26", 1.85),
}

def generate_zone_dataset(zone_id: str) -> pd.DataFrame:
    timestamps = pd.date_range(START, END, freq="h")
    demand_base = ctgan_model.sample(len(timestamps))  # CTGAN base distribution

    # Apply MULTIPLICATIVE seasonality (replaces additive — panel-resolved)
    demand_base["demand_units"] *= festival_multiplier(timestamps)
    demand_base["demand_units"] *= salary_week_boost(timestamps)

    # Inject external regressors (IMD API + HTP — Arjun's compute-feasible fallback)
    weather = fetch_weather_3hourly(zone_id, START, END).resample("h").interpolate()
    demand_base["precipitation_mm"] = weather["precip"]
    demand_base["temperature_c"]    = weather["temp"]
    demand_base["traffic_profile_index"] = load_htp(zone_id, timestamps)  # HTP fallback

    demand_base["zone_id"] = zone_id
    return demand_base

all_zones = pd.concat([generate_zone_dataset(z) for z in ZONES])
# Final shape: ~3,940,000 rows × 12 columns
```

Statistical Validation Gates — M7

TEST	THRESHOLD	SCOPE	FAILURE ACTION
TSTR Ratio	`≥ 0.90`	Full dataset vs golden holdout	Block shadow mode — regenerate with corrected multipliers
Diwali WMAPE	`< 22%`	Festival window ±7 days	Increase festival window coverage; recalibrate multipliers
Overall WMAPE	`< 15%`	Full 3-year holdout	Extend CTGAN training epochs; review base demand distribution
KS p-value (demand dist.)	`> 0.05` (gate: `< 0.01` → halt)	Per-zone demand CDF comparison	Pipeline halt at p < 0.01; regenerate with +100 CTGAN epochs
Correlation Delta (regressors)	`max Δ < 0.15`	Correlation matrix vs real holdout	Review feature engineering constraints; check IMD interpolation
Tier Isolation	`Leak count = 0`	Train/test split	SHA-256 re-hash; audit trip_id overlap

EXPERT PANEL — M7/M1 DEPENDENCY DEBATE (ABRIDGED)

PRIYA (Urban) *"We should use H3-grid 15-minute fleet telemetry for real traffic regressors — this would dramatically improve zone-level accuracy."*

ARJUN (MLOps) *"Packet loss rates and compute costs make H3 telemetry at that granularity infeasible for production. Historical Traffic Profiles give us 80% of the signal at 10% of the cost."*

VIKRAM (Demand) *"Accepted — HTP fallback it is for traffic. We get festival multipliers and weather from external APIs; that combination should get us to the 80% threshold. And disconnect M1 from M7 until we're through shadow validation."*

PART II — MODEL SPECIFICATIONS

P1 HIGH MODEL 1

ISS — Inventory Scheduling System

*Current: 75.0% · Target: 88.0%+ · Delta: **+13pp** · Upstream dependency: M7 (currently blocked)*

Interim Operational State

ISS (M1) is currently in a co-dependency failure loop with Prophet. The model cannot reach its 88%+ target while receiving a corrupted 60%-reliable upstream prior. The expert panel resolved this with a clean architectural decision: **hard decouple M1 from M7 output** until M7 passes shadow validation, and substitute a statistically sound interim prior.

CURRENT STATE (BLOCKED)

ISS receives Prophet output as the demand prior. Prophet at 60% reliability means ISS begins every scheduling cycle with a fundamentally unreliable input distribution. No amount of ISS-specific training data improvement can overcome a corrupted upstream prior.

INTERIM STATE (ACTIVE NOW)

Naive 4-week rolling average + festival multiplier as the ISS prior. Simple, statistically transparent, and operationally superior to a 60% Prophet output. This fallback is maintained until M7 achieves `TSTR ≥ 0.90`.

Naive Baseline Specification

```
# M1 ISS — Naive Prior Implementation
# Interim solution while M7 undergoes shadow mode rebuilding
# Source: Vikram (Demand Lead) panel resolution

def naive_iss_prior(zone_id: str, target_timestamp: pd.Timestamp) -> float:
    """
    4-week rolling average with festival multiplicative adjustment.
    Replaces M7 Prophet output during shadow mode period.
    """
    historical_window = get_demand_history(
        zone_id=zone_id,
        end=target_timestamp - pd.Timedelta(days=1),
        weeks=4
    )
    rolling_avg = historical_window["demand_units"].mean()

    # Festival multiplier from calendar lookup
    festival_mult = get_festival_multiplier(target_timestamp)
    salary_mult   = get_salary_week_boost(target_timestamp)

    return rolling_avg * festival_mult * salary_mult

# Circuit breaker: automatically activates this function when M7 TSTR < 0.90
# See Section 08 — Circuit Breaker & Fallback Patterns for full CI/CD integration
```

ISS-Specific Dataset Requirements

FEATURE NAME	TYPE	DESCRIPTION	STATUS
demand_prior	Float	M7 forecast output (blocked) OR naive_iss_prior() (interim)	✅ Interim Active
inventory_level	Float	Current stock level per zone (internal warehouse feed)	✅ Available
festival_multiplier	Float [0.8–5.0]	Shared with M7 dataset — payday + festival calendar	⚠️ Shared with M7 build
lead_time_hours	Integer	Supplier lead time per SKU category — zone-variable	✅ Available
stockout_flag	Boolean (target)	Historical stockout events — ISS learns to predict and prevent	⚠️ Needs 3yr historical audit

⚠️ M1 UNLOCK CONDITION

M1 (ISS) dataset for prophet-integrated mode **cannot be finalized** until M7 exits shadow mode with `TSTR ≥ 0.90`. The ISS team should proceed with naive-prior training data preparation now but hold the M7-integrated dataset version until the upstream gate clears.

PART II — MODEL SPECIFICATIONS

P1 HIGH MODELS 2 & 3

Fraud Detection — GPS Spoofing & Trip Fraud

*Current: 85.0% · Target: 92.0%+ · Delta: **+7pp** · No upstream dependency*

The Telemetry Collection Dilemma — Panel Resolution

Neha's initial specification of 2 million full telemetry pings (with IMU correlation and battery temperature data) was technically sound but ran into three hard constraints: **legal** (continuous IMU collection without explicit consent is problematic under Indian data protection frameworks), **storage** (2M high-resolution pings at 40 fields each exceeds practical storage budgets), and **timeline** (collection would take 14+ weeks to accumulate organically).

NEHA (T&S) *"We need 2 million pings with full IMU correlation and battery temperature. GPS spoofing detection requires physical sensor cross-validation — there is no shortcut."*

KARTHIK (Infra) *"2M full pings is a storage and timeline blocker. And continuous IMU logging without consent is a legal risk we can't absorb."*

ARJUN IYER (Net) *"Network Trust Score: Timing Advance, Cell-ID consistency, RSRQ jitter, handoff patterns. If Trust_Score is high, GPS is almost certainly legitimate. Trigger high-res telemetry only when the score drops below threshold."*

NEHA (T&S) ✓ *"If Trust_Score < 0.7, trigger the High-Resolution Telemetry Snapshot. That's the compromise. 500,000 shadow trips with conditional snapshots. Plus 1,000 manually adjudicated for golden holdout."*

PANEL RESOLUTION

- `500,000` shadow trips (low-overhead)
- Conditional high-res snapshots when `Trust_Score < 0.7`
- `1,000` manually adjudicated golden holdout
- `8%` fraud contamination rate preserved

Dataset Volume Specification

TIER	VOLUME	COLLECTION METHOD	PURPOSE
Shadow Trips (base)	`500,000 trips`	Passive logging (network metadata only)	Base training distribution; covers normal + edge-case routes
High-Res Snapshots	Triggered subset (~5–15% of shadow trips)	IMU + Battery Temp snapshot when Trust_Score < 0.7	Ground-truth signal for fraud classifiers; consent-bounded
Golden Holdout	`1,000 cases`	Manual adjudication by Trust & Safety analysts	TSTR gate — synthetic data must match on this set
Synthetic Augmentation	CTGAN + SMOTE-Tomek (8% fraud ratio)	SDV synthetic generation from real shadow trip distributions	Class balance and edge case coverage (spoofing scenarios)

Network Trust Score Feature Engineering

FEATURE NAME	TYPE	SOURCE	GPS SPOOFING INDICATOR LOGIC	STATUS
timing_advance	Float [0–1282]	LTE/5G network stack	TA inconsistency with claimed GPS location → spoofing signal	✅ Available
cell_id_consistency	Float [0–1]	Cell tower handoff logs	Sudden cell tower impossibility (teleportation) → hard spoofing flag	✅ Available
rsrq_jitter	Float	LTE RSRQ measurement reports	Abnormally stable RSRQ → spoofed point-source signal (no multipath)	⚠️ Needs extraction pipeline
handoff_pattern_score	Float [0–1]	Network event logs	Handoff sequence inconsistent with route speed/direction	⚠️ Needs extraction pipeline
network_trust_score	Float [0–1]	Composite (above 4 features)	Weighted ensemble; threshold 0.7 triggers high-res snapshot	⚠️ Needs model training
imu_accel_mismatch	Float (conditional)	High-res snapshot (Trust_Score < 0.7)	Accelerometer vector inconsistent with claimed velocity/direction	⚠️ Conditional collection
battery_temp_c	Float (conditional)	High-res snapshot (Trust_Score < 0.7)	Abnormal battery temp for claimed route duration/load	⚠️ Conditional collection
snr_variability	Float	GPS chipset logs	Low SNR variability = suspicious (spoofed signal lacks multipath)	❌ Requires chipset API access
velocity_jump_flag	Boolean	GPS ping sequence	Sudden velocity impossibility between consecutive pings	✅ Available
fraud_label	Binary (target)	Manual adjudication (1,000 golden set)	Ground truth: 0=legitimate, 1=fraud (GPS spoof OR trip fraud)	⚠️ Requires analyst adjudication

```
# Conditional Telemetry Snapshot Trigger — M2/M3 Data Collection
# Source: Arjun Iyer (Network) + Neha (T&S) panel resolution

def collect_trip_telemetry(trip_id: str, network_signal: dict) -> dict:
    # Compute Network Trust Score from low-overhead network metadata
    trust_score = compute_trust_score(
        timing_advance=network_signal["ta"],
        cell_id_delta=network_signal["cell_id_changes"],
        rsrq_jitter=network_signal["rsrq_std"],
        handoff_score=network_signal["handoff_consistency"]
    )

    base_record = {
        "trip_id": sha256_hash(trip_id),  # anonymized via SHA-256 + salt
        "network_trust_score": trust_score,
        "velocity_jump_flag": detect_velocity_jump(trip_id),
        "timestamp": pd.Timestamp.utcnow().isoformat()
    }

    # Conditional high-resolution snapshot — ONLY when Trust_Score < 0.7
    # Legally bounded: triggered only on suspicious signal, not universal collection
    if trust_score < 0.70:
        hires_snapshot = {
            "imu_accel_mismatch": get_imu_accel_delta(trip_id),
            "battery_temp_c":   get_battery_temp(trip_id),
            "snapshot_trigger": "trust_score_below_threshold"
        }
        base_record.update(hires_snapshot)

    return base_record  # written to shadow_trips table
```

PART II — MODEL SPECIFICATIONS

P3 STANDARD MODEL 4

NLP — Customer Support Classifier

*Current: 90.0% · Target: 93.0%+ · Delta: **+3pp** · Latency constraint: sub-15ms hot-path*

The Sarcasm Problem — Latency vs Accuracy Tradeoff

M4's remaining 7pp accuracy gap is dominated by two phenomena: **code-switching** (Tanglish, Hinglish — mid-sentence language transitions) and **sarcasm** (negative sentiment expressed as positive language, e.g., "Great, my package arrived only 3 weeks late!"). Siddharth proposed real-time telemetry-sentiment cross-checking using IndicBERT + phonetic embeddings, but this approach clashed with the 15ms latency budget for the alert hot-path.

SIDDHARTH (NLP) *"For sarcasm, we need real-time telemetry cross-checks — IndicBERT with phonetic embeddings can catch the code-switching, and delivery telemetry confirms or refutes the expressed sentiment."*

KARTHIK (Real-time) *"We have a sub-15ms initial alert latency budget on the hot-path. A full IndicBERT inference with telemetry joins will never fit in that window."*

SIDDHARTH ✓ *"Distilled ONNX student model on the hot-path for initial classification. Asynchronous telemetry enrichment — up to 500ms — for sarcasm confirmation and model update. The corpus corpus needs 50,000 utterances to train both."*

ARCHITECTURE DECISION

Two-stage inference:

- Stage 1: ONNX student model, <15ms, hot-path
- Stage 2: Full IndicBERT + telemetry join, async, ≤500ms, sarcasm confirmation

Corpus Specification

METRIC	SPECIFICATION	NOTES
Total Utterances	`50,000` anonymized support utterances	Panel specification — minimum for stable code-switching coverage
Language Mix	Tamil, Hindi, English, Tanglish, Hinglish	Phonetic embeddings required for transliterated text
Edge Cases	`500 rows` curated sarcasm/irony edge cases	Previous spec; maintained as critical validation set
PII Handling	PII-scrubbed before corpus ingestion	Phone numbers, order IDs, names — regex + NER scrubbing pipeline required
Label Schema	Multi-class intent + sentiment + sarcasm_flag	Sarcasm label added as explicit target field for async confirmation path
Inference Target (Stage 1)	`<15ms` P99 latency (ONNX student, hot-path)	Quantized INT8 ONNX export; benchmarked on prod hardware
Inference Target (Stage 2)	`≤500ms` async enrichment window	Full IndicBERT + delivery telemetry join for sarcasm confirmation

```
# M4 NLP — Two-Stage Inference Pipeline
# Stage 1: ONNX distilled student (hot-path, <15ms)
# Stage 2: Async telemetry enrichment (sarcasm confirmation, ≤500ms)

import onnxruntime as ort
from asyncio import create_task

student_session = ort.InferenceSession("models/nlp_student_int8.onnx")

async def classify_support_utterance(ticket_id: str, text: str) -> dict:
    # STAGE 1: Hot-path inference (must complete <15ms)
    phonetic_emb = get_phonetic_embedding(text)  # handles Tanglish/Hinglish
    stage1_result = student_session.run(
        output_names=["intent", "sentiment", "sarcasm_prob"],
        input_feed={"phonetic_embedding": phonetic_emb}
    )

    # Immediately write Stage 1 result to alert system
    await emit_alert(ticket_id, stage1_result, source="stage1_student")

    # STAGE 2: Async sarcasm confirmation — does NOT block hot-path
    # Triggers if sarcasm_prob > 0.4 in Stage 1 (borderline cases)
    if stage1_result["sarcasm_prob"] > 0.40:
        create_task(
            async_sarcasm_confirm(ticket_id, text, stage1_result)  # ≤500ms budget
        )

    return stage1_result

async def async_sarcasm_confirm(ticket_id: str, text: str, stage1: dict):
    # Full IndicBERT inference + delivery telemetry join
    indicbert_result = run_indicbert(text)
    delivery_telemetry = await fetch_delivery_telemetry(ticket_id)  # actual delivery data
    confirmed = cross_validate_sentiment(indicbert_result, delivery_telemetry)
    if confirmed != stage1["sentiment"]:
        await update_ticket_classification(ticket_id, confirmed, source="stage2_async")
```

PART II — MODEL SPECIFICATIONS

P2 MEDIUM MODEL 5

Blackout Predictor

80.0% → 90.0%+ · Delta: **+10pp**

M5 predicts driver/vehicle availability blackouts (supply disruptions). Core failure mode: insufficient representation of extreme weather events and festival-period driver withdrawal patterns in the training set.

KEY DATASET REQUIREMENTS

Volume	`24 months` hourly blackout events
Features	Weather severity, festival calendar, local events, driver tenure
Edge Cases	Heatwave >40°C, monsoon >50mm/hr, Diwali withdrawal
Balance	SMOTE-Tomek — blackout events are rare (class imbalance)

CRITICAL FEATURE ADDITION

weather_severity_index: Composite of IMD precipitation + temperature + wind — primary predictor of mass blackout events that current model cannot anticipate.

P2 MEDIUM MODEL 6

Traffic Flow Model

88.0% → 93.0%+ · Delta: **+5pp**

M6 routes deliveries based on real-time traffic state predictions. The model lacks encoding for Chennai Metro Phase 2 construction disruptions — a systematic and predictable bottleneck currently invisible to the training data.

CHENNAI METRO DISRUPTION FEATURES

metro_construction_zone	OMR SRP-Navalur, Sholinganallur, Velachery
corridor_blockage_factor	0.0–1.0 per corridor segment
post_launch_multiplier	30–40% travel time reduction (future state)
diversion_active_flag	Velachery alternate route activation

DATA SOURCE

Chennai Metro Phase 2 construction maps (Corridor 3 on OMR, Corridor 5 near Velachery) + CMRL schedule feeds. 1,000 rows specifically for Chennai traffic validation included in existing dataset baseline.

M6 CHENNAI TRAFFIC VALIDATION SPECIFICATION

Baseline (Existing)	New: Construction Phase	Future State Labels
`1,000 rows` Chennai traffic validation dataset — maintained from prior spec	`+2,000 rows` Metro Phase 2 disruption scenarios per bottleneck corridor	30–40% travel time delta labeled for post-Metro-launch model version preparation

---

PART III — STATISTICAL QUALITY FRAMEWORK / SECTION 04

TSTR Validation Protocol

Train on Synthetic, Test on Real — the production deployment gate for every model.

TSTR DEFINITION

TSTR measures whether a model trained entirely on synthetic data performs comparably when evaluated on real-world holdout data. The **TSTR Utility Gap** = (Real-data performance − Synthetic-trained performance) / Real-data performance. A gap below 10% (TSTR ratio ≥ 0.90) indicates the synthetic dataset faithfully captures the real distribution. A gap above 10% triggers pipeline halt and dataset regeneration.

```
# TSTR Implementation — Full Protocol
# Gate: TSTR ratio must be ≥ 0.90 for all models before production deployment
# Source: Arjun (MLOps Lead) + statistical validation literature

import numpy as np
from sklearn.metrics import mean_absolute_percentage_error, f1_score
from scipy.stats import ks_2samp

def compute_tstr_ratio(
    synthetic_train_data: pd.DataFrame,
    real_holdout_data: pd.DataFrame,
    model_class,
    target_col: str,
    metric_fn = mean_absolute_percentage_error
) -> float:
    """
    TSTR Protocol:
    1. Train model on synthetic data
    2. Evaluate on real holdout
    3. Compare to real-data-trained baseline
    Returns: TSTR ratio (target ≥ 0.90)
    """
    X_synth = synthetic_train_data.drop(columns=[target_col])
    y_synth = synthetic_train_data[target_col]
    X_real  = real_holdout_data.drop(columns=[target_col])
    y_real  = real_holdout_data[target_col]

    # Train on synthetic, evaluate on real (TSTR)
    model_synth = model_class().fit(X_synth, y_synth)
    perf_tstr   = metric_fn(y_real, model_synth.predict(X_real))

    # Baseline: train on real, evaluate on real (upper bound)
    model_real = model_class().fit(X_real, y_real)
    perf_real  = metric_fn(y_real, model_real.predict(X_real))

    # TSTR ratio: how close is synthetic-trained performance to real-trained?
    tstr_ratio = 1.0 - ((perf_tstr - perf_real) / perf_real)
    return tstr_ratio  # gate: must be ≥ 0.90

def run_full_tstr_gate(model_id: str, synth_df: pd.DataFrame, real_df: pd.DataFrame):
    ratio = compute_tstr_ratio(synth_df, real_df, ...)
    if ratio < 0.90:
        raise PipelineHaltException(
            f"TSTR gate FAILED for {model_id}: ratio={ratio:.3f} (required ≥ 0.90)"
        )
    log_metric("tstr_ratio", ratio, tags={"model": model_id})
    return ratio
```

Full Statistical Gate Matrix — All Models

GATE	METRIC / TEST	PASS THRESHOLD	HARD-HALT TRIGGER	FAILURE ACTION	APPLIES TO
G-1	TSTR Utility Ratio	`≥ 0.90`	`< 0.85` (emergency halt)	Block deployment; regenerate synthetic data; review feature constraints	All 7 models
G-2	KS Test (distribution match)	`p > 0.05`	`p < 0.01` → pipeline halt	Regenerate with +100 CTGAN epochs; review sampler hyperparameters	All 7 models
G-3	Chi-Square (categorical distributions)	`p > 0.05`	`p < 0.01` → pipeline halt	Resample categorical features; audit CTGAN conditional vectors	M2/M3, M4
G-4	Correlation Delta (max)	`max Δ < 0.15`	`Δ > 0.25` on any feature pair	Review feature engineering constraints; check interpolation artifacts	M7, M1, M6
G-5	Tier Isolation (train/test leakage)	`Leak count = 0`	Any leak detected → halt	SHA-256 re-hash of trip_ids; full split audit; see Section 06	All 7 models
G-6	Fraud Contamination Rate	`8.0% ± 0.5%`	Rate < 6% or > 11%	Re-run SMOTE-Tomek; adjust oversampling ratio	M2/M3 only
G-7	Diwali WMAPE (M7 specific)	`< 22%`	`> 30%`	Recalibrate festival multipliers; extend Diwali window ±3 days	M7 only
G-8	NLP Sarcasm Precision (async)	`≥ 0.85 precision`	`< 0.70`	Expand sarcasm edge-case corpus; review IndicBERT fine-tuning	M4 only

Automated KS / Chi-Square Pipeline

```
# Automated distribution validation — runs on every synthetic dataset commit
# Integrated into CI/CD pre-deployment check (GitHub Actions step: validate-distributions)

from scipy.stats import ks_2samp, chi2_contingency
import pandas as pd

def validate_distributions(
    synth_df: pd.DataFrame,
    real_df: pd.DataFrame,
    numeric_cols: list[str],
    categorical_cols: list[str],
    halt_threshold: float = 0.01,
    warn_threshold: float = 0.05
) -> dict:
    results = {"passed": True, "warnings": [], "halts": []}

    # KS Test: numeric feature distributions
    for col in numeric_cols:
        stat, p_val = ks_2samp(synth_df[col].dropna(), real_df[col].dropna())
        if p_val < halt_threshold:
            results["halts"].append({"col": col, "test": "KS", "p": p_val})
            results["passed"] = False
        elif p_val < warn_threshold:
            results["warnings"].append({"col": col, "test": "KS", "p": p_val})

    # Chi-Square Test: categorical feature distributions
    for col in categorical_cols:
        synth_counts = synth_df[col].value_counts().sort_index()
        real_counts  = real_df[col].value_counts().sort_index()
        contingency  = pd.concat([synth_counts, real_counts], axis=1).fillna(0)
        chi2, p_val, _, _ = chi2_contingency(contingency.values)
        if p_val < halt_threshold:
            results["halts"].append({"col": col, "test": "Chi2", "p": p_val})
            results["passed"] = False

    # Correlation Delta: check inter-feature correlation matrix drift
    synth_corr = synth_df[numeric_cols].corr()
    real_corr  = real_df[numeric_cols].corr()
    delta_max  = (synth_corr - real_corr).abs().max().max()
    if delta_max > 0.15:
        results["warnings"].append({"test": "CorrelationDelta", "max_delta": delta_max})
    if delta_max > 0.25:
        results["halts"].append({"test": "CorrelationDelta", "max_delta": delta_max})
        results["passed"] = False

    return results
```

PART III — STATISTICAL QUALITY FRAMEWORK / SECTION 06

Tier Isolation — SHA-256 Data Lineage Protocol

Zero-leakage enforcement between training, validation, and test splits across all seven models.

WHY THIS IS NON-NEGOTIABLE

Any data leakage between the synthetic training corpus and the real-world golden holdout set invalidates the TSTR gate entirely. A model that has "seen" holdout data during training will report inflated TSTR ratios, producing false deployment confidence. The SHA-256 hashing protocol eliminates this by creating a cryptographically verifiable, collision-resistant identifier for every training sample — making cross-tier leakage detectable at any point in the pipeline.

```
# SHA-256 Tier Isolation Protocol
# Arjun (MLOps): "If Prophet's reliability is tanking, the pipe to ISS shuts off."
# Same logic: if tier isolation fails, the entire pipeline shuts off."

import hashlib, secrets
from functools import partial

TIER_SALT = secrets.token_hex(32)  # generated once per dataset version; stored in secrets vault

def compute_tier_hash(trip_id: str, tier: str) -> str:
    """
    tier: one of 'train' | 'validation' | 'test' | 'golden_holdout'
    Produces a unique, collision-resistant identifier per (trip_id, tier) pair.
    """
    payload = f"{TIER_SALT}:{tier}:{trip_id}".encode("utf-8")
    return hashlib.sha256(payload).hexdigest()

def audit_tier_isolation(
    train_hashes: set[str],
    holdout_hashes: set[str]
) -> int:
    """Returns leak count — must equal 0 for gate G-5 to pass."""
    leaks = train_hashes.intersection(holdout_hashes)
    if len(leaks) > 0:
        raise TierIsolationViolation(
            f"{len(leaks)} trip_ids found in both training and holdout tiers. "
            f"Halt all downstream model training. Re-hash and re-split dataset."
        )
    return 0  # gate G-5 passed

def assign_tier(trip_id: str, split_ratios: dict = {"train": 0.70, "val": 0.15, "test": 0.15}) -> str:
    # Deterministic assignment — same trip_id always maps to same tier
    hash_int = int(hashlib.md5(f"{TIER_SALT}:{trip_id}".encode()).hexdigest(), 16)
    bucket   = (hash_int % 100) / 100.0
    if bucket < split_ratios["train"]:
        return "train"
    elif bucket < split_ratios["train"] + split_ratios["val"]:
        return "validation"
    else:
        return "test"
```

TRAIN TIER	VALIDATION TIER	GOLDEN HOLDOUT (REAL)
70%	**15%**	**15%**
Synthetic + augmented data. No real-world samples.	Synthetic held-out. Used for hyperparameter tuning only.	Real-world data. TSTR gate computed here. Never exposed to train tier.

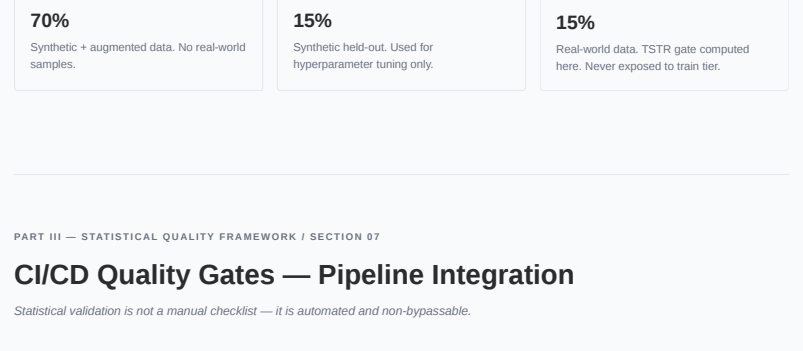
PART III — STATISTICAL QUALITY FRAMEWORK / SECTION 07

CI/CD Quality Gates — Pipeline Integration

Statistical validation is not a manual checklist — it is automated and non-bypassable.

> *"If a model trained on synthetic data doesn't hit ≥90% of its target metric on a 'Golden' real-world holdout set, the deployment is automatically killed. No exceptions, no overrides."*
>
> — Arjun (MLOps & Data Quality Lead), Expert Panel Discussion

DEPLOYMENT PIPELINE — GATE SEQUENCE



```
# GitHub Actions CI/CD Integration — .github/workflows/validate-dataset.yml
# Every dataset commit triggers this workflow before any model training job

name: Dataset Quality Gates

on:
  push:
    paths: ['datasets/**', 'feature_engineering/**']
  pull_request:
    branches: ['main', 'staging']

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - name: Tier Isolation Audit
        run: python scripts/audit_tier_isolation.py --model-id $MODEL_ID
        # Gate G-5: exit code 1 if any leak detected → workflow fails

      - name: Distribution Validation (KS + Chi-Sq + Corr Delta)
        run: python scripts/validate_distributions.py --halt-threshold 0.01
        # Gates G-2/G-3/G-4: auto-halts on p < 0.01 or Corr Delta > 0.25

      - name: TSTR Gate
        run: python scripts/run_tstr_gate.py --model-id $MODEL_ID --threshold 0.90
        # Gate G-1: exit code 1 if TSTR ratio < 0.90 → blocks all downstream jobs

      - name: Fraud Contamination Check
        run: python scripts/check_fraud_ratio.py --target 0.08 --tolerance 0.005
        if: env.MODEL_ID == 'M2' || env.MODEL_ID == 'M3'
        # Gate G-6: fraud contamination must be 8% ± 0.5%

      - name: Post Results to Slack
        if: always()
        run: python scripts/notify_validation_result.py --channel ml-deployments
```

AUTOMATION PRINCIPLE

No human sign-off can override a failed statistical gate. The CI/CD system is the arbiter. This removes the risk of "it's good enough" deployment decisions under production pressure — a failure mode that has historically caused silent model degradation in production ML systems.

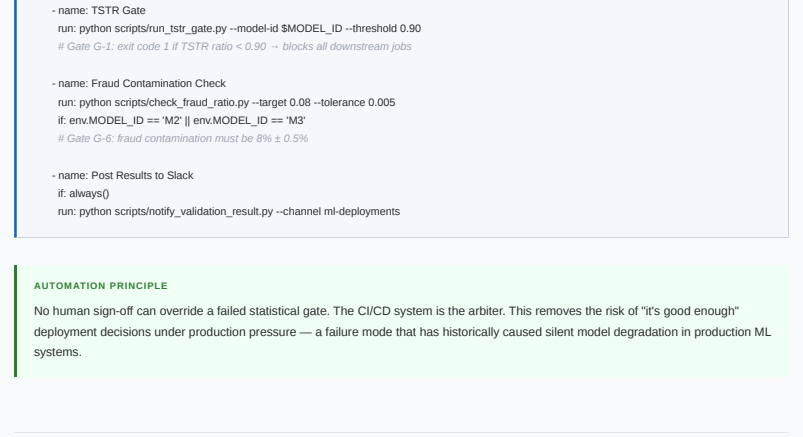
PART IV — OPERATIONAL BLUEPRINTS / SECTION 08

Circuit Breaker & Fallback Patterns

Production resilience through automated fallback — the system degrades gracefully rather than failing silently.

> *"If Prophet's reliability is tanking, the pipe to ISS shuts off and it falls back to Vikram's Naive Baseline. The system should never pass a known-bad prediction downstream."*
>
> — Arjun (MLOps & Data Quality Lead), Expert Panel Discussion

CIRCUIT BREAKER STATE MACHINE — M7 → M1 DEPENDENCY



```
# Circuit Breaker Implementation — M7 → M1 Dependency
# Monitors TSTR gate in real-time; switches M1 prior source automatically

from enum import Enum
from dataclasses import dataclass

class BreakerState(Enum):
    CLOSED    = "closed"     # M7 → M1 active, TSTR passing
    OPEN      = "open"       # M7 blocked, naive prior active
    HALF_OPEN = "half_open"  # M7 in shadow mode, TSTR re-evaluation

@dataclass
class ProphetCircuitBreaker:
    state: BreakerState = BreakerState.OPEN  # starts OPEN — M7 currently at 60%
    tstr_threshold: float = 0.90
    shadow_evaluation_window: int = 14       # days of shadow output to evaluate

    def get_m1_demand_prior(self, zone_id: str, ts: pd.Timestamp) -> float:
        if self.state == BreakerState.CLOSED:
            return prophet_forecast(zone_id, ts)      # M7 output — trusted
        else:
            return naive_iss_prior(zone_id, ts)       # fallback — always available

    def evaluate_shadow_mode(self, tstr_ratio: float) -> BreakerState:
        if self.state == BreakerState.HALF_OPEN:
            if tstr_ratio >= self.tstr_threshold:
                self.state = BreakerState.CLOSED
                notify_oncall("M7 validated — reconnecting to M1 pipeline")
            else:
                self.state = BreakerState.OPEN
                notify_oncall(f"M7 shadow TSTR={tstr_ratio:.3f} — remaining in fallback")
        return self.state

breaker = ProphetCircuitBreaker()  # singleton — injected into M1 inference service
```

GENERALISATION — ALL UPSTREAM DEPENDENCIES

The same circuit breaker pattern applies to any upstream→downstream model dependency in the pipeline. The ProphetCircuitBreaker class above is a template — instantiate with the relevant model pair and TSTR threshold. All breaker state transitions are logged to the observability stack with timestamps for post-incident audit.

PART IV — OPERATIONAL BLUEPRINTS / SECTION 09

Conditional Telemetry Snapshot Strategy

Privacy-preserving high-resolution data collection for M2/M3 fraud detection — triggered only on genuine suspicion signals.

Network Trust Score Architecture

The Network Trust Score is a lightweight composite score computed from four low-overhead network metadata fields available on every trip without any additional collection infrastructure. The score runs continuously with negligible compute cost, triggering expensive high-resolution collection only when a trip crosses the `Trust_Score < 0.7` threshold.

COMPONENT	WEIGHT	SPOOFING SIGNAL
timing_advance	0.30	TA inconsistent with GPS claim
cell_id_consistency	0.30	Impossible cell tower jump
rsrq_jitter	0.25	Too-stable signal = point source
handoff_pattern	0.15	Handoff sequence mismatch

COLLECTION DECISION FLOW

Trust_Score ≥ 0.7
Log network metadata only. No IMU. No battery temp. Trip proceeds normally. Low storage footprint.

↓

Trust_Score < 0.7
Trigger High-Res Snapshot: IMU + Accel correlation + Battery Temp. Driver notified per consent framework. Snapshot queued for adjudication.

↓

Manual Adjudication Queue
1,000 cases/quarter reviewed by Trust & Safety analysts. Labelled outcomes feed golden holdout for TSTR gate G-1.

LEGAL & PRIVACY NOTE

High-resolution IMU and battery telemetry collection must be explicitly covered in the driver app's terms of service and data collection notice. The conditional trigger (Trust_Score < 0.7) must be disclosed as a fraud-prevention mechanism. Legal counsel should review the consent framework before the 500,000 shadow trip collection begins. This was a hard requirement raised by Karthik during the panel discussion and accepted by Neha.

PART IV — OPERATIONAL BLUEPRINTS / SECTION 10

Async Enrichment Pattern — NLP Sarcasm Path

Maintaining sub-15ms hot-path latency while enabling full telemetry-enriched sarcasm confirmation for M4.

STAGE 1 — HOT PATH (<15MS)

- ONNX INT8 quantized student model
- Phonetic embeddings (Tanglish/Hinglish safe)
- Outputs: intent + sentiment + sarcasm_prob
- Immediately writes alert to downstream system
- No blocking on telemetry joins

STAGE 2 — ASYNC CONFIRMATION (≤500MS)

- Full IndicBERT inference (teacher model)
- Delivery telemetry join (actual vs claimed)
- Sarcasm confirmation or correction
- Triggers only when sarcasm_prob > 0.40
- Updates ticket classification if divergent from Stage 1

PARAMETER	STAGE 1 (HOT PATH)	STAGE 2 (ASYNC)
Latency Budget	`< 15ms P99`	`≤ 500ms`
Model	ONNX INT8 student (distilled IndicBERT)	Full IndicBERT (teacher)
Features	Phonetic embedding only	Phonetic + delivery telemetry cross-join
Output	Preliminary classification	Confirmed/corrected classification
Trigger Condition	Every incoming support ticket	sarcasm_prob > 0.40 in Stage 1
Corpus Required	50,000 utterances (full corpus)	500 sarcasm edge cases (subset)
Infra Pattern	Synchronous — blocks alert emission	Async task (asyncio) — does not block

---

PART IV — OPERATIONAL BLUEPRINTS / SECTION 11

External Data Source Integration Guide

Authoritative external sources that provide real-world signal for synthetic dataset regressors.

WEATHER REGRESSORS

IMD Mausam API

India Meteorological Department

Granularity	3-hourly (interpolate to hourly)
Fields	precip_mm, temp_c, wind_kmh
Coverage	All major Indian cities; historical portal at IMD Pune
Key Event	NE Monsoon: Chennai Oct–Dec (flag separately)
Models Using	M7 (Prophet), M5 (Blackout)

TEMPORAL REGRESSORS

Indian Festival & Payday Calendar

Compiled from official calendar + HR survey data

Diwali 2024	`2024-10-31` · multiplier 4.2–4.8×
Diwali 2025	`2025-10-20` · for forward validation
IPL Window	`2024-03-22 to 2024-05-26` · evening 1.8×
Corporate Payday	1st–5th monthly · spike 1.4×
Informal Payday	7th–10th monthly · spike 1.25×
Models Using	M7, M1, M5

TRAFFIC REGRESSORS

Chennai Metro Phase 2 — CMRL Data

Chennai Metro Rail Limited construction maps + schedules

Corridor 3	OMR: SRP Tools → Navalur (active disruption)
Corridor 5	Near Velachery (active diversion routes)
Sholinganallur	Junction blockage — major bottleneck
Post-Launch	30–40% travel time reduction (label future state)
Models Using	M6 (Traffic), M7 (traffic regressor)

FRAUD SIGNAL REFERENCE

GPS Spoofing Detection — Physical Validation

Cross-validated against GPS chipset physics literature

Urban Canyon	SNR highly variable; satellite count drops legitimately
Spoofing Signal 1	Sudden velocity jump between consecutive pings
Spoofing Signal 2	IMU/accelerometer mismatch with claimed velocity
Spoofing Signal 3	Physically impossible multi-satellite point-source SNR (too stable)
Models Using	M2, M3 (Fraud Detection)

IMPLEMENTATION ROADMAP

Phased Implementation Timeline

Prioritised execution sequence with resource requirements and milestone gates.

01 PHASE 1 — WEEKS 1–6 · P0 CRITICAL PATH

Prophet Dataset Rebuild + Circuit Breaker Deployment

WEEK	DELIVERABLE	OWNER	GATE
1–2	IMD Mausam API integration + historical pull (3yr)	Data Eng	API access confirmed
2–3	Festival/payday/IPL calendar feature engineering	Vikram	Feature spec review
3–4	HTP traffic profile generation (150 zones)	Priya + Arjun	Correlation Delta < 0.15
4–5	CTGAN training on base demand + regressor injection	Vikram Rao	KS p > 0.05 per zone
5–6	TSTR gate on golden holdout; shadow mode launch	Arjun (MLOps)	`TSTR ≥ 0.90`
6	Circuit breaker deployment; M1 naive fallback activation	Karthik	Zero-downtime deploy

Phase 1 Success Criteria: M7 achieves `WMAPE < 15%` overall and `Diwali WMAPE < 22%` on the 3-year holdout. TSTR ratio ≥ 0.90. Circuit breaker in HALF-OPEN state ready for reconnection evaluation.

02 PHASE 2 — WEEKS 4–16 · P1 PARALLEL TRACK

Fraud Detection Dataset + NLP Corpus

WEEK	DELIVERABLE	OWNER	GATE
4–6	Network Trust Score pipeline (TA + Cell-ID + RSRQ + Handoff)	Arjun Iyer	Feature validation on 10k trips
5–8	500k shadow trip collection (consent framework live)	Neha + Legal	Legal sign-off required
6–10	NLP corpus collection — 50k anonymised utterances	Siddharth	PII scrub pipeline audited
8–12	CTGAN + SMOTE-Tomek fraud dataset (8% contamination)	Vikram Rao	Gate G-6: 8% ± 0.5%
10–14	1,000 golden holdout manual adjudication	Neha (T&S analysts)	100% adjudication complete
12–16	ONNX student distillation + hot-path latency validation	Karthik	`<15ms P99`

03 PHASE 3 — WEEKS 8–20 · P2 SCHEDULED TRACK

Blackout + Traffic Dataset Completion

WEEK	DELIVERABLE	OWNER	GATE
8–12	Chennai Metro Phase 2 construction disruption dataset (+2k rows)	Priya	CMRL data access confirmed
10–14	Blackout predictor dataset — 24-month hourly events	Arjun (MLOps)	SMOTE-Tomek class balance verified
14–18	M6 traffic model TSTR validation on Chennai holdout	Arjun (MLOps)	`TSTR ≥ 0.90`
16–20	M5 blackout TSTR + full CI/CD gate integration	Karthik	All 8 gates passing

ONGOING — QUARTERLY VALIDATION CADENCE

After initial deployment, each model's golden holdout set is refreshed quarterly with 500–1,000 consented real-world samples. This prevents distribution drift from eroding TSTR ratios over time. The CI/CD pipeline runs the full gate sequence on each quarterly refresh automatically.

Fraud (M2/M3): 250 new adjudicated cases per quarter + red-team adversarial collection	**NLP (M4):** 500 new utterances per quarter; sarcasm edge cases curated by Siddharth's team	**Prophet (M7):** IMD API historical pull refreshed; festival calendar updated for next 12 months

---

RISK MANAGEMENT

Risk Register

Principal uncertainties and mitigation strategies identified during expert panel review.

RISK-01 HIGH — **M7 Shadow Mode Validation Exceeds 6-Week Timeline**

Scenario: CTGAN fails to capture festival demand multiplier distribution correctly even after regressor injection, requiring multiple regeneration cycles beyond the 6-week Phase 1 budget.

Mitigation: Naive 4-week rolling average + festival multiplier (Vikram's fallback) is production-viable for up to 12 weeks without significant business impact. M1 accuracy with naive prior is estimated at ~78% — below target but operationally acceptable during the transition window. Parallel dataset experimentation (varying CTGAN epoch counts and conditional vectors) should begin at Week 2 to accelerate iteration cycles.

RISK-02 MEDIUM — **Legal Delay on Conditional Telemetry Consent Framework**

Scenario: Legal review of the IMU/battery telemetry conditional snapshot collection takes longer than estimated, blocking M2/M3 shadow trip collection from starting on schedule.

Mitigation: Begin network metadata-only collection (Trust Score components) immediately — this requires no additional consent beyond standard app terms. Full telemetry snapshots commence only after legal clearance. The 1,000-case golden holdout adjudication (which uses existing fraud cases from internal Trust & Safety records) can proceed independently to maintain the TSTR gate timeline.

RISK-03 MEDIUM — **IMD API Access / Historical Data Quality Gaps**

Scenario: IMD Mausam API historical data has coverage gaps for specific cities or time windows, creating missing regressor values in the Prophet dataset that cannot be imputed without introducing distribution artifacts.

Mitigation: Identify a commercial weather data fallback (e.g., Open-Meteo, Visual Crossing) that provides equivalent 3-hourly historical coverage. Budget for commercial API access as a contingency line item. For gaps < 72 hours, linear interpolation is statistically acceptable. For gaps > 72 hours, flag the time window and exclude from WMAPE computation — do not impute over extended missing periods.

RISK-04 LOW-MED — **NLP Corpus Sarcasm Coverage Insufficient for 93% Target**

Scenario: 50,000 utterances provide adequate general intent coverage but the sarcasm-labelled subset (drawn from organic support tickets) is too sparse to train the async confirmation path to ≥ 0.85 precision.

Mitigation: Supplement organic sarcasm cases with adversarially generated sarcastic utterances, validated by native Tanglish/Hinglish speakers. Siddharth's team should maintain a curated 500-case sarcasm seed set (from the original spec) as the minimum baseline. The async enrichment path design ensures sarcasm errors are recoverable — Stage 1 misclassifications are corrected within 500ms without customer-visible impact.

APPENDIX A

Full Feature Dictionaries

Complete column specifications for all seven models — reference documentation for implementation teams.

P0 · MODEL 7

Prophet — Complete Feature Dictionary

COLUMN NAME	DTYPE	RANGE / LEVELS	NULL POLICY	NOTES
timestamp	datetime64[ns, UTC]	2022-01-01 to 2024-12-31	Never null	Hourly frequency; UTC+5:30 for India
zone_id	str (H3 hex)	150 unique values	Never null	H3 resolution 8 hex identifiers
demand_units	float32	[0, ∞)	Never null	Target variable; CTGAN synthetic base
festival_multiplier	float32	[0.80, 5.00]	Default 1.0	Multiplicative; see festival calendar
salary_week_flag	int8	{0, 1, 2}	Default 0	0=normal, 1=corporate, 2=informal
ipl_match_flag	bool	{True, False}	Default False	Evening hours only on match days
precipitation_mm	float32	[0, 200]	Interpolate <72h gaps	IMD Mausam API; 3h→1h interpolation
temperature_c	float32	[5, 50]	Interpolate <72h gaps	Heatwave flag at >40°C
traffic_profile_index	float32	[0.0, 1.0]	Never null	HTP fallback; zone × hour lookup
hour_sin	float32	[-1.0, 1.0]	Never null	sin(2π × hour/24) cyclical encoding
hour_cos	float32	[-1.0, 1.0]	Never null	cos(2π × hour/24) cyclical encoding
day_of_week	int8	[0, 6]	Never null	0=Monday; weekend suppression applied

P1 · MODELS 2 & 3

Fraud Detection — Complete Feature Dictionary

COLUMN NAME	DTYPE	RANGE / LEVELS	NULL POLICY	NOTES
trip_id_hash	str (SHA-256)	64-char hex	Never null	SHA-256(SALT + "shadow" + raw_trip_id)
timing_advance	float32	[0, 1282]	Never null	LTE TA units; 0=co-located, 1282=max range
cell_id_consistency	float32	[0.0, 1.0]	Never null	1.0=consistent; 0.0=impossible jump
rsrq_jitter	float32	[0, 30]	Nullable if extraction unavailable	High = multipath (real); low = point-source (spoof)
handoff_pattern_score	float32	[0.0, 1.0]	Nullable	1.0=consistent with route; 0.0=impossible
network_trust_score	float32	[0.0, 1.0]	Never null	Weighted composite; gate at 0.7
velocity_jump_flag	bool	{True, False}	Never null	True = physically impossible velocity between pings
imu_accel_mismatch	float32	[0.0, 1.0]	Null if Trust >= 0.7	Conditional — only collected in high-res snapshot
battery_temp_c	float32	[20, 60]	Null if Trust >= 0.7	Conditional — only collected in high-res snapshot
fraud_label	int8	{0, 1}	Null for shadow; required for golden	Target; 0=legitimate, 1=fraud; 8% contamination
adjudication_confidence	float32	[0.0, 1.0]	Null for shadow	Analyst confidence in golden holdout labels only

P3 · MODEL 4

NLP Classifier — Complete Feature Dictionary

COLUMN NAME	DTYPE	RANGE / LEVELS	NULL POLICY	NOTES
ticket_id_hash	str (SHA-256)	64-char hex	Never null	PII-safe identifier; raw ticket_id never stored
utterance_scrubbed	str	Max 512 tokens	Never null	PII-scrubbed; phone/order/name removed via regex+NER
language_detected	str	{en, hi, ta, mixed}	Never null	Automatic language detection; mixed = Tanglish/Hinglish
phonetic_embedding	float32[768]	L2-normalised	Never null	IndicBERT phonetic embed; handles transliteration
intent_label	str (categorical)	12 intent classes	Never null	Primary classification target for Stage 1
sentiment_label	str	{positive, neutral, negative}	Never null	Stage 1 output; may be corrected by Stage 2
sarcasm_flag	bool	{True, False}	Null for non-edge-case corpus	Labelled in 500-case sarcasm edge set + Stage 2 confirmations
stage2_sentiment	str	{positive, neutral, negative, null}	Null if Stage 2 not triggered	Async-confirmed sentiment; overrides stage1 if divergent
delivery_delta_hours	float32	[-∞, ∞]	Null if Stage 2 not triggered	Actual vs promised delivery gap; used in sarcasm validation

---

APPENDIX B

Full Dependency Graph Summary

All seven models with data flow relationships, priority levels, and dataset linkages.

MODEL	PRIORITY	CURRENT	TARGET	UPSTREAM INPUTS	DOWNSTREAM CONSUMERS	DATASET SIZE	GENERATION METHOD
M7 Prophet	P0	`60.0%`	`80.0%+`	IMD API, Festival Calendar, HTP	M1 (ISS) — blocked until TSTR ≥ 0.90	~3.9M rows	CTGAN + real regressor injection
M1 ISS	P1	`75.0%`	`88.0%+`	M7 Prophet (blocked) → Naive prior (interim)	Route dispatch, inventory allocation	Shared with M7	Naive baseline + festival multiplier
M2 Fraud	P1	`85.0%`	`92.0%+`	Network Trust Score, GPS pings, conditional IMU	Fraud alert system	500k shadow + 1k golden	CTGAN + SMOTE-Tomek (8% fraud)
M3 Fraud	P1	`85.0%`	`92.0%+`	Shared dataset with M2 (trip-level features)	Fraud alert system	Shared with M2	CTGAN + SMOTE-Tomek (8% fraud)
M4 NLP	P3	`90.0%`	`93.0%+`	Support ticket corpus; delivery telemetry (async)	Support triage system; escalation routing	50k utterances	Organic corpus + adversarial sarcasm augmentation
M5 Blackout	P2	`80.0%`	`90.0%+`	IMD weather, festival calendar, driver tenure	Supply planning, surge pricing	24 months hourly events	SMOTE-Tomek (rare event balancing)
M6 Traffic	P2	`88.0%`	`93.0%+`	HTP, CMRL Metro construction maps, IMD weather	Route optimisation, ETA engine	1k (baseline) + 2k (Metro Phase 2)	Real data + synthetic Metro disruption scenarios

---

APPENDIX C

Glossary of Technical Terms

Reference definitions for all specialised terminology used in this specification.

TSTR (Train on Synthetic, Test on Real)
A synthetic data validation protocol that measures whether a model trained exclusively on synthetic data generalises to real-world holdout data. TSTR Ratio = 1 − (performance_gap / real_baseline). Gate threshold: ≥ 0.90.

CTGAN (Conditional Tabular GAN)
A generative adversarial network architecture designed for tabular data synthesis. Handles mixed data types (continuous + categorical) and imbalanced distributions. Used as the primary synthetic base generator for M7, M2, and M3 datasets.

SMOTE-Tomek
A combined over- and under-sampling technique. SMOTE synthesises minority class examples via interpolation; Tomek link removal cleans boundary noise from the majority class. Used for the 8% fraud contamination target and M5 blackout event balancing.

KS Test (Kolmogorov-Smirnov)
A non-parametric test that compares two continuous probability distributions. In this context, it measures whether the synthetic feature distribution matches the real-world reference distribution. Gate: p-value > 0.05 (warn), p-value < 0.01 (halt).

WMAPE (Weighted MAPE)
Weighted Mean Absolute Percentage Error. Weights the error by actual demand magnitude, preventing low-demand periods from dominating the accuracy metric. Primary evaluation metric for M7 Prophet. Target: < 15% overall, < 22% on Diwali festival windows.

HTP (Historical Traffic Profile)
Zone-level × hour-of-day congestion index derived from historical GPS traces. Accepted as the compute-feasible fallback regressor for M7 after Arjun blocked Priya's H3 15-minute telemetry proposal on packet loss and cost grounds.

Network Trust Score
A lightweight composite fraud-risk score computed from four LTE network metadata fields (Timing Advance, Cell-ID consistency, RSRQ jitter, handoff patterns). Proposed by Arjun Iyer as the privacy-preserving trigger for conditional high-resolution telemetry collection. Threshold: < 0.7 triggers snapshot.

IndicBERT
A multilingual BERT-variant pre-trained on 12 Indian languages including Tamil and Hindi. Used as the teacher model in M4's two-stage inference architecture. Full inference is too slow for the hot-path (<15ms) but operates within the 500ms async enrichment budget for sarcasm confirmation.

ONNX (Open Neural Network Exchange)
An open format for representing ML models enabling interoperability between frameworks. In M4, the IndicBERT student model is exported to INT8-quantised ONNX format for deployment on the hot-path inference server to achieve sub-15ms P99 latency.

Circuit Breaker Pattern
A software resilience pattern that prevents a failing upstream component from degrading downstream systems. Three states: CLOSED (normal operation), OPEN (upstream blocked, fallback active), HALF-OPEN (shadow mode validation in progress). Applied to the M7→M1 dependency.

Golden Holdout
A real-world validation dataset kept completely separate from synthetic training data, used exclusively for TSTR gate evaluation. Never used for model training or hyperparameter tuning. For fraud models: 1,000 manually adjudicated cases. Refreshed quarterly with new consented samples.

Tier Isolation
The strict enforcement of zero data overlap between training, validation, and test/holdout tiers via SHA-256 content-addressed hashing. A trip_id appearing in both train and golden holdout constitutes a tier isolation violation and halts the entire deployment pipeline.

QUICK REFERENCE CARD

One-Page Summary — All 7 Models

UNIVERSAL QUALITY GATES — APPLY TO ALL MODELS

≥ 0.90	> 0.05	< 0.15	= 0
TSTR RATIO (G-1)	KS P-VALUE (G-2)	CORRELATION Δ MAX (G-4)	TIER ISOLATION LEAKS (G-5)

P0 · M7	P1 · M1	P1 · M2	P1 · M3	P3 · M4	P2 · M5	P2 · M6
Prophet	ISS	Fraud	Fraud	NLP	Blackout	Traffic
80%+	88%+	92%+	92%+	93%+	90%+	93%+
3.9M rows WMAPE <15%	Naive prior now active	500k shadow 8% fraud ratio	Shared M2 dataset	50k utterances <15ms P99	24mo hourly SMOTE-Tomek	1k+2k rows Metro Phase 2

DOCUMENT AUTHORITY — EXPERT PANEL SIGN-OFF

Vikram Demand Forecasting Lead M7 Prophet · M1 ISS	**Neha** Trust & Safety Architect M2/M3 Fraud Detection	**Siddharth** Indic NLP Expert M4 NLP Classifier	**Arjun** MLOps & Data Quality Lead All models · CI/CD gates
Priya Subramanian Urban Mobility Analyst M6 Traffic · M7 Geography	**Karthik Nair** MLOps Engineering (Real-time) Latency · Circuit breakers	**Arjun Iyer** Network Performance Engineer Network Trust Score · M2/M3	**Vikram Rao** Synthetic Data Scientist CTGAN · SMOTE-Tomek · All datasets

This document reflects the consensus outcomes of the Hustlr ML Expert Panel discussion. All dataset specifications, validation thresholds, and architectural decisions are panel-endorsed.

ML-DATASET-BLUEPRINT-v1.0
INTERNAL — ENGINEERING USE ONLY

THREE GOVERNING PRINCIPLES OF THIS BLUEPRINT

Data quality is the new model architecture

Improving training distributions produces more reliable gains than architectural changes. Fix the data first, then tune the model.

Break the dependency, unblock the pipeline

The M7→M1 critical path is the system's binding constraint. Resolving it — even with a naive fallback — unblocks the highest-impact reliability improvement available.

Automate the quality gates, not the judgement

Statistical gates (TSTR, KS, Chi-square, Tier Isolation) must be enforced by CI/CD, not human sign-off. Human judgement belongs in dataset design and adjudication — not deployment approval.

HUSTLR PLATFORM — ML DATASET ENGINEERING BLUEPRINT · ML-DATASET-BLUEPRINT-V1.0

END OF DOCUMENT · 7 MODELS · 8 QUALITY GATES · 8 EXPERT CONTRIBUTORS